

**LAPORAN PRAKTIKUM CHAT APPLICATION DENGAN IPC
OPERASI SYSTEM**



Dosen Pengampu :

Triawan Adi Cahyanto M.Kom

Disusun Oleh :

HAFABI ARROHIM (2410651013)

PROGRAM STUDY TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

TAHUN 2025

1. Tujuan

Tujuan dari percobaan ini adalah untuk memahami konsep proses dan cara proses saling berkomunikasi menggunakan metode *Inter Process Communication (IPC), khususnya melalui **Shared Memory* dan *Semaphore* di dalam sistem operasi.

2. Dasar Teori

- Konsep Proses

Proses adalah program yang sedang dieksekusi. Setiap proses memiliki ruang alamat sendiri dan dapat berjalan bersamaan dengan proses lain. Sistem operasi bertugas mengatur jalannya proses-proses ini agar tidak saling mengganggu.

- IPC (Inter Process Communication) Methods

IPC adalah mekanisme yang memungkinkan proses untuk saling bertukar data. Beberapa metode IPC antara lain:

- Shared Memory: Proses berbagi area memori yang sama untuk komunikasi.
- Semaphore: Proses berbagi area memori yang sama untuk komunikasi.
- Message Passing: Proses saling bertukar pesan melalui sistem operasi.

3. Percobaan

a. Source Code Lengkap

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <unistd.h>

#define MAX_MSG 20
#define MSG_LEN 100

struct Chat {
    int count;
    char messages[MAX_MSG][MSG_LEN];
};

// Operasi semaphore
void sem_wait_op(int semid) {
    struct sembuf op = {0, -1, 0};
    semop(semid, &op, 1);
}

void sem_signal_op(int semid) {
    struct sembuf op = {0, 1, 0};
    semop(semid, &op, 1);
}

int main() {
    key_t key = 1234; // key unik untuk shared memory dan semaphore
    int shmid = shmget(key, sizeof(struct Chat), IPC_CREAT | 0666);
    int semid = semget(key, 1, IPC_CREAT | 0666);
```

```

struct Chat *chat = (struct Chat *) shmat(shmid, NULL, 0);

// Inisialisasi semaphore jika baru dibuat
semctl(semid, 0, SETVAL, 1);

int choice;
char msg[MSG_LEN];

while (1) {
    printf("\n=== CHAT ROOM ===\n");
    printf("1. Kirim pesan\n");
    printf("2. Lihat history\n");
    printf("3. Exit\n");
    printf("Pilih: ");
    scanf("%d", &choice);
    getchar(); // buang newline

    if (choice == 1) {
        printf("Masukkan pesan: ");
        fgets(msg, MSG_LEN, stdin);
        msg[strcspn(msg, "\n")] = 0; // hapus newline

        sem_wait_op(semid);
        if (chat->count < MAX_MSG) {
            strcpy(chat->messages[chat->count], msg);
            chat->count++;
        } else {
            printf("Pesan penuh!\n");
        }
    }
}

```

```

        msg[strcspn(msg, "\n")] = 0; // hapus newline

        sem_wait_op(semid);
        if (chat->count < MAX_MSG) {
            strcpy(chat->messages[chat->count], msg);
            chat->count++;
        } else {
            printf("Pesan penuh!\n");
        }
        sem_signal_op(semid);

    } else if (choice == 2) {
        sem_wait_op(semid);
        printf("\n--- HISTORY PESAN ---\n");
        for (int i = 0; i < chat->count; i++) {
            printf("%s\n", chat->messages[i]);
        }
        sem_signal_op(semid);

    } else if (choice == 3) {
        printf("Keluar...\n");
        break;
    } else {
        printf("Pilihan tidak valid.\n");
    }
}

shmdt(chat);
return 0;
}

```

b. Output program

User 1 di dalam terminal 1

```
3. Exit
Pilih: Pilihan tidak valid.

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: 1
Masukkan pesan: Bocah Bagus

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: ^Z
[2]+  Stopped                  ./chat_ipc
fabyy@fabyyar-bieee:~$ ./chat_ipc

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: 1
Masukkan pesan: Anak Muda

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: |
```

User 2 di dalam terminal 2

```
=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: 2

--- HISTORY PESAN ---
Halo Dunia

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: 2

--- HISTORY PESAN ---
Halo Dunia
Bocah Bagus

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih: 2

--- HISTORY PESAN ---
Halo Dunia
Bocah Bagus
Anak Muda

=== CHAT ROOM ===
1. Kirim pesan
2. Lihat history
3. Exit
Pilih:
```

c. Analisis Hasil

Berdasarkan hasil output, program berhasil membuat tiga proses anak yang menjalankan tugas berbeda:

Proses	Tugas	Output	Status
--------	-------	--------	--------

Child 1	Sorting array	Menampilkan hasil urutan angka	Selesai
---------	---------------	--------------------------------	---------

Child 2	Searching angka	Menampilkan hasil pencarian	Selesai
---------	-----------------	-----------------------------	---------

Child 3	Perhitungan (sum)	Menampilkan hasil penjumlahan	Selesai
---------	-------------------	-------------------------------	---------

Parent	Koordinasi proses anak	Menampilkan waktu eksekusi total	Selesai
--------	------------------------	----------------------------------	---------

Setiap proses memiliki PID unik, menunjukkan bahwa proses dijalankan terpisah.

Parent menunggu seluruh child selesai melalui fungsi `waitpid()`, sesuai konsep synchronization antar proses.

d. Jawaban tugas

- Program berhasil membuat multiple child processes.
- Setiap child menjalankan tugas berbeda (sorting, searching, calculation).
- Parent mengumpulkan hasil dan menampilkan PID, waktu, dan status.

e. Analisis Performa

Waktu eksekusi total menunjukkan efisiensi koordinasi antar proses.

Karena setiap proses sederhana, waktu eksekusinya kecil (< 0.01 detik).

Namun, dalam sistem nyata, pembuatan proses tambahan menambah overhead CPU dan memori

6. Kesimpulan

Program berhasil mensimulasikan sistem manajemen proses dengan tiga child process yang menjalankan tugas berbeda. Fungsi `fork()` dan `waitpid()` bekerja untuk membuat dan menyinkronkan proses. Hasil menunjukkan setiap proses memiliki PID unik dan status “selesai”. Praktikum ini membantu memahami dasar proses paralel dan koordinasi antar proses (IPC) di sistem operasi.